



Università
Ca'Foscari
Venezia

Into to Learning

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

What is Learning?

The word “learning” has several different meanings

- ▶ Memorizing something
- ▶ Learning facts through observation and exploration
- ▶ Improving motor/cognitive skills through practice
- ▶ Organizing new knowledge into general, effective representations

One of the most common kinds of learning is the acquisition of information with the goal of making predictions about the future.

But what exactly gives us license to imagine we can predict the future?

Induction

Lots of philosophers have thought about this problem. David Hume first framed the problem of induction as follows:



David Hume
(1711–1776)

- ▶ Piece of bread number 1 was nourishing when I ate it.
- ▶ Piece of bread number 2 was nourishing when I ate it.
- ▶ Piece of bread number 3 was nourishing when I ate it.
- ▶ Piece of bread number 100 was nourishing when I ate it.
- ▶ Therefore, all pieces of bread will be nourishing if I eat them.

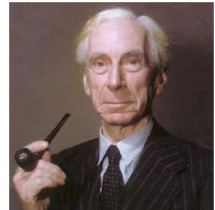
When and why is it okay to apply inductive reasoning??

Induction

If asked why we believe the sun will rise tomorrow, we shall naturally answer, 'Because it has always risen every day.' We have a firm belief that it will rise in the future, because it has risen in the past.

The real question is: Do any number of cases of a law being fulfilled in the past afford evidence that it will be fulfilled in the future?

It has been argued that we have reason to know the future will resemble the past, because what was the future has constantly become the past, and has always been found to resemble the past, so that we really have experience of the future, namely of times which were formerly future, which we may call past futures. But such an argument really begs the very question at issue. We have experience of past futures, but not of future futures, and the question is: Will future futures resemble past futures?



Bertrand Russell
(1872–1970)



Induction

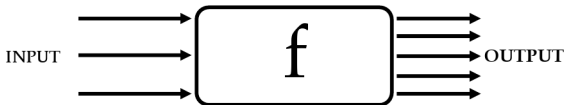
We won't worry too much about this problem.

If induction is not, somehow, justified, then we have no reason to get out of bed in the morning let alone study Artificial Intelligence!

Keep in mind, however, that all the knowledge that is not inborn or printed out on tables by God, had to be learned through observation

Learning from Examples

Learning from examples: goal is not to memorize but to generalize, eg predict.



Given a set of l examples (data)

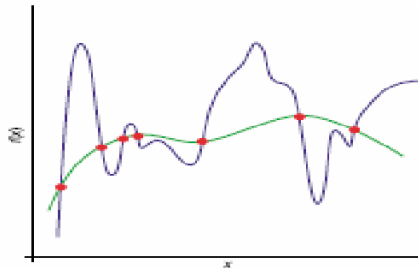
$$\{(x_1, y_1), (x_2, y_2), \dots, (x_l, y_l)\}$$

Question: find function f such that is a good predictor of y for a future input x (fitting the data is not enough!):

Choice of function

Which function would you choose for the given data points?

- = data from f
- = function f
- = approximation of f

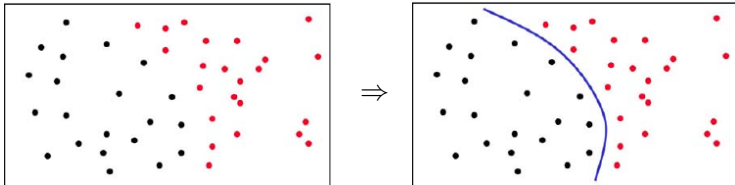


Learning is all about generalization: estimating value of function where there are no data

Types of Learning:

Supervised Learning

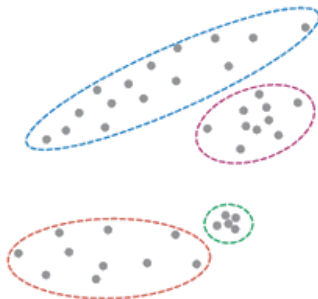
Supervised learning given a set of example input/output pairs find a function that does a good job of predicting the output associated to a new input



Types of Learning:

Unsupervised Learning

Clustering (unsupervised learning) given a set of examples, but no labeling to them, group the examples into “natural” groups





Types of Learning:

Reinforcement Learning

Reinforcement learning an agent interacting with the environment, makes observations, takes actions, and is rewarded or punished; it should learn to choose actions in such a way as to obtain a lot of reward

The Learning Problem

Assume

- ▶ $\mathcal{X}$ – Set of possible inputs
- ▶ $\mathcal{Y}$ – Set of possible outputs
- ▶ $X : \Omega \rightarrow \mathcal{X}$ $Y : \Omega \rightarrow \mathcal{Y}$ – Random variables
- ▶ $\mu(X, Y)$ – **Unknown** joint distribution

A **training set** is a set of samples

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_l, y_l)\}$$

iid drawn from μ

Independent and Identical distribution

The Learning Problem

The Hypothesis $\mathcal{H}$ is a space of functions $f : X \rightarrow Y$

A **learning algorithm** is a map

$$L : (\mathcal{X} \times \mathcal{Y})^n \rightarrow \mathcal{H}$$

that looks at S and selects from $\mathcal{H}$ a function f_S such that

$$f_S(x) \approx y$$

in a **predictive** way

Generalization

An informal definition of generalization is to find a model that predicts well new data

What are the conditions ensuring generalization?

We need a more formal definition...

Let V be a **loss function**, i.e., a function indicating the penalty for a wrong prediction

The **expected** or **true error** of prediction function f is

$$I[f] = \mathbb{E} V[f, z] = \int \mu(z) V(f, z) dz$$

with $z = (x, y)$

The expected error is the expectation of the loss incurred on a new example drawn at random from μ

Generalization

The **empirical error** is

$$l_S[f] = \frac{1}{l} \sum_{i=1}^l V(f, z_i)$$

The empirical error is the error observed in the training set S

A very natural requirement for f_S is **distribution independent generalization**

$$\forall \mu, l_S[f_S] \xrightarrow{P} l[f_S] \quad \text{or} \quad \forall \mu, \eta, \lim_{l \rightarrow \infty} P(\{|l_S[f_S] - l[f_S]| > \eta\}) = 0$$

In other words, the training error for the solution must converge to the expected error and thus be a “proxy” for it.

Otherwise the solution would not be “predictive”

Well-Posedness

A superficially different requirement for learning is well-posedness of the associated optimization problem

A problem is well-posed if its solution

- ▶ exists
- ▶ is unique
- ▶ is stable, eg depends continuously on the data



J. S. Hadamard
(1865–1963)

Well-Posedness

It is possible to show that under quite general assumptions generalization and well-posedness are equivalent, e.g. one implies the other.

Stability implies generalization:

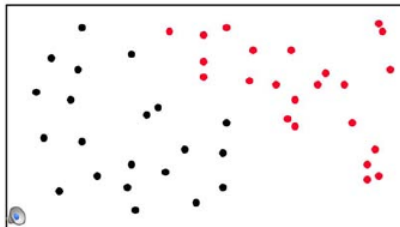
- ▶ a stable solution is predictive
- ▶ for a special case (Empirical Risk Minimization) also viceversa

Control of Complexity

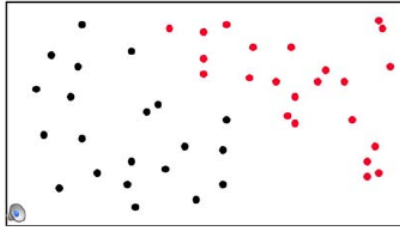
The problem does not have a predictive solution in general (just fitting the data does not work).

Choosing an appropriate hypothesis space H (for instance a compact set of continuous functions) can guarantee generalization (how good depends on the problem and other parameters)

Imagine that you were given all these points, and you needed to guess a function of their x, y coordinates that would have one output for the red ones and a different output for the black ones.

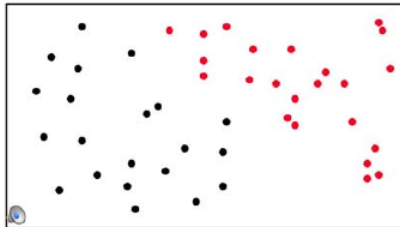


Control of Complexity

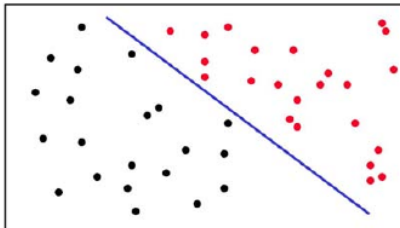


What is the right hypothesis?

Control of Complexity

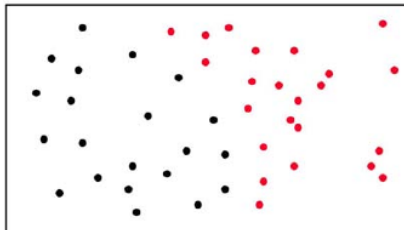


What is the right hypothesis?



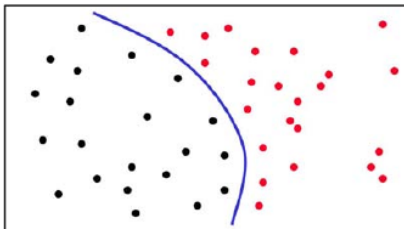
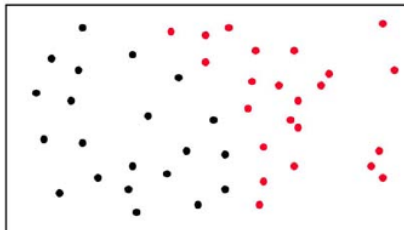
Control of Complexity

Now, what is the right hypothesis?



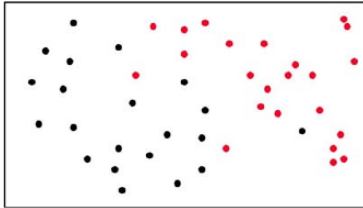
Control of Complexity

Now, what is the right hypothesis?



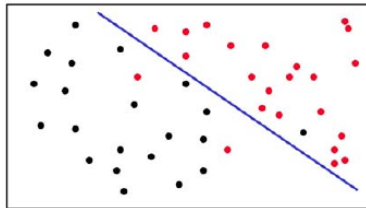
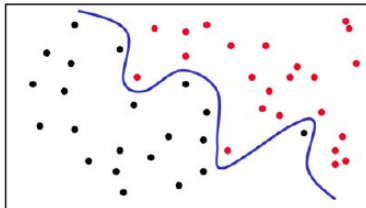
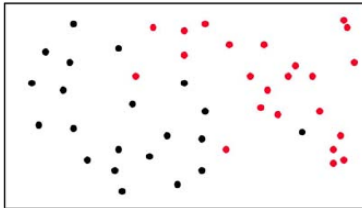
Control of Complexity

What about now?



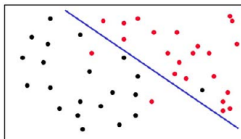
Control of Complexity

What about now?

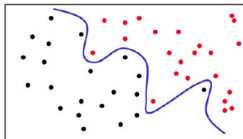


Bias Vs Variance

If we separate it with a line, we have low complexity, but lots of error.



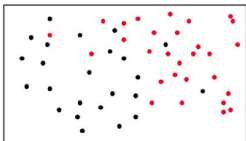
If we separate it with a squiggly snake, we can reduce the error but add a lot of complexity.



What is the problem with complexity?

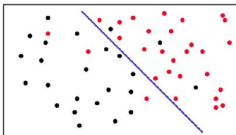
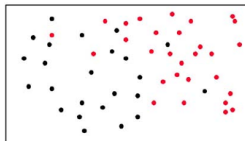
Resampling the Training Set

Let us add some points drawn from the same distribution

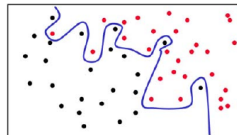


Resampling the Training Set

Let us add some points drawn from the same distribution



The linear hypothesis does not
need to change much



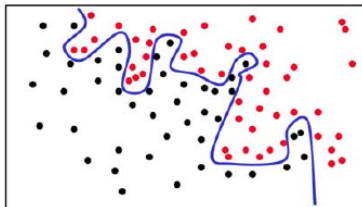
The snake would have to be
changed radically

Not well posed and not general!

More Data!

In practice, it is common to allow hypotheses of increased complexity as the amount of available training data increases.

In this situation, we might feel justified in choosing a moderately complex hypothesis.



Ockham's Razor

Hypotheses should:

Do a good job of describing the data

- ▶ Ideally $f(x_i) = y_i$
- ▶ Loss function $V(x_i, y_i)$

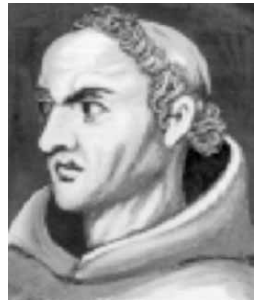
Not be too complex

- ▶ How do we measure complexity?

Ockham's Razor in machine learning encourages simplicity, guiding us to choose the least complex model that provides a good fit.

Learning methods differ in terms of

- ▶ **the form** of the hypothesis
- ▶ **the way** the computer finds an hypothesis given the data



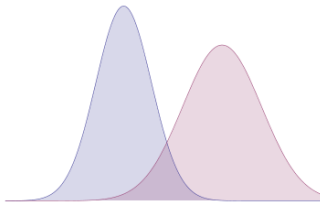
William of Ockham
(XIV century)

By reducing unnecessary complexity, we improve the likelihood that the model will generalize well to new data.

Choosing a Loss Function

How do we choose a loss function?

In a classification problem where the two classes have the distributions pictured below, how do we choose to which class to assign an input?



If you want to minimize the *expected classification errors*, then one would assign x to the class with the highest density $\mu_C(x)$ (Bayes decision rule)

Choosing a Loss Function

But, Is the number of errors always the correct choice?

The cost of a mistake can be very different for the two classes:

- ▶ consider the case of a Doctor deciding whether to give or not a lifesaving drug that has very little side-effects.

A short survey

Consider the following two games:

Game 1 Someone arrives and offers you two alternatives:

1. a sure gain of EUR 250
2. a 25% chance of winning EUR 1000 and a 75% chance of winning nothing

Which one would you prefer?

A short survey

Consider the following two games:

Game 1 Someone arrives and offers you two alternatives:

1. a sure gain of EUR 250
2. a 25% chance of winning EUR 1000 and a 75% chance of winning nothing

Which one would you prefer?

Game 2 The next day the same guy offers you two alternatives:

1. a sure loss of EUR 750
2. a 75% chance of losing EUR 1000 and a 25% chance of losing nothing

Which one would you prefer now?

How well does it work?

Error on the training data is not a good indicator of performance on future data

- ▶ New data will probably not be exactly the same as the training data!
- ▶ Overfitting – fitting the training data too precisely - usually leads to poor results on new data

The generalization performance of a machine learning method relates to its prediction capability on independent test sets.

Assessment of this performance is extremely important in practice, since it guides the choice of the machine learning method or model.

Model Assessment

Training Error

- ▶ The average loss over the training samples

Test Error

- ▶ The average error that results from using a machine learning method to predict the response on a new observation
- ▶ The **prediction error** over an **independent test sample**

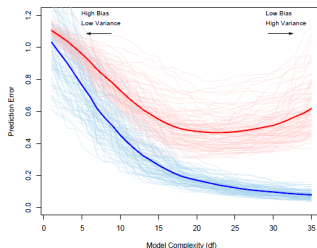
Model Assessment

As the model becomes more and more complex, it uses the training data more and is able to adapt to more complicated underlying structures.

► low bias/high variance

Training error consistently decreases with model complexity.

A model with zero training error is overfit to the training data and will typically generalize poorly.

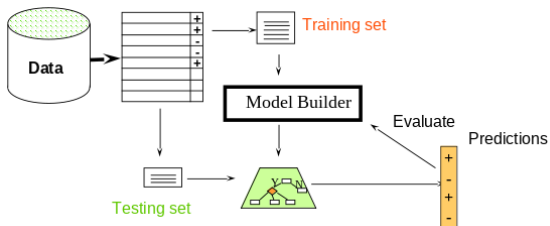


Evaluation on Large Data

If many samples are available, randomly split data into **training** and **test** sets

- ▶ larger training set $\Rightarrow$ better classifier (but returns diminish)
- ▶ larger test set $\Rightarrow$ more accurate the error estimate
- ▶ usually 2/3 for train, 1/3 for test

Build a classifier using the train set and evaluate it using the test set.



Parameter Tuning

It is important that the test data is not used in any way to create the classifier

Some learning schemes operate in two stages:

- ▶ Stage 1: builds the basic structure
- ▶ Stage 2: optimizes parameter settings

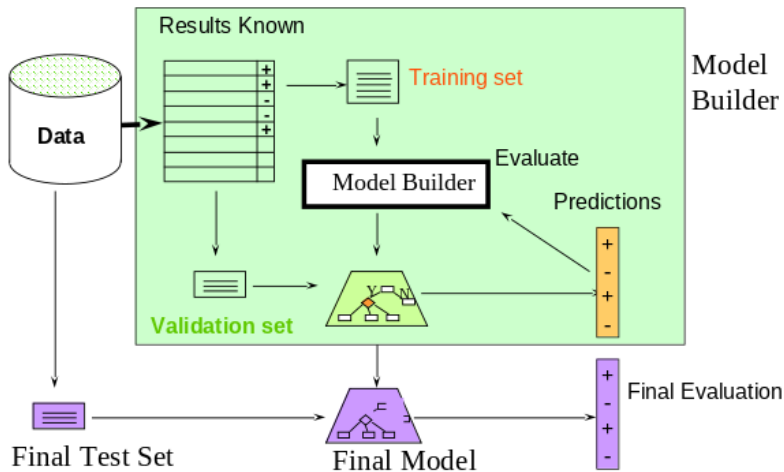
The test data can't be used for parameter tuning!

Proper procedure uses three sets: training data, validation data, and test data

- ▶ Validation data is used to optimize parameters

Once evaluation is complete, all the data (training + validation) can be used to build the final classifier

Train – Validation – Test



Handling Unbalanced Data

Sometimes, classes have very unequal frequency

- ▶ Attrition prediction: 97% stay, 3% attrite (in a month)
- ▶ medical diagnosis: 90% healthy, 10% disease
- ▶ eCommerce: 99% don't buy, 1% buy
- ▶ Security: >99.99% of people are not terrorists

Majority class classifier can be 97% correct, but useless

Handling Unbalanced Data

Sometimes, classes have very unequal frequency

- ▶ Attrition prediction: 97% stay, 3% attrite (in a month)
- ▶ medical diagnosis: 90% healthy, 10% disease
- ▶ eCommerce: 99% don't buy, 1% buy
- ▶ Security: >99.99% of people are not terrorists

Majority class classifier can be 97% correct, but useless

Build **BALANCED** train and test sets, and train model on a balanced set

- ▶ randomly select desired number of minority class instances
- ▶ add equal number of randomly selected majority class

What Performance are we Estimating?

The error on the test set evaluates the performance of a given predictor

It doesn't work to evaluate the performance of the learning algorithm

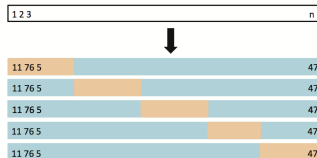
To estimate the (expected) error of the algorithm we must average over several instances of generated classifiers

The most common approach is to perform cross-validation

Can also be used to perform model (parameter) selection

Leave-One-Out K-Fold Cross-Validation

We **RANDOMLY** divide the data set of into K folds (typically $K = 5$ or 10)



The first fold is treated as a validation set, and the method is fit on the remaining $K - 1$ folds.

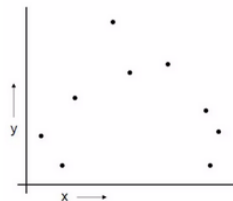
The MSE is computed on the observations in the held-out fold.

The process is repeated K times, taking out a different part each time.

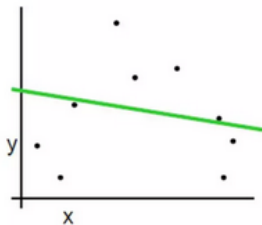
By averaging the K estimates of the test error, we get an estimated validation (test) error rate for new observations.

Assessment Example

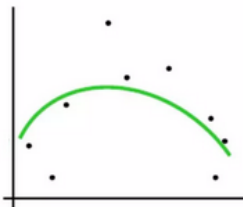
Find best best model for the following data



3 hypotheses spaces (classes of models)



Linear



Quadratic

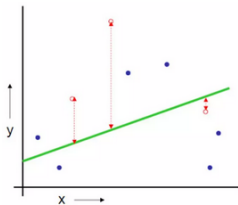


Join the dots

Test Set

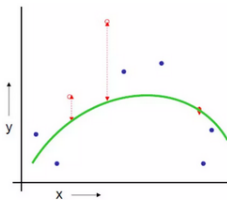
Randomly chose 30% of the data to be in a test set

The remainder is the training set



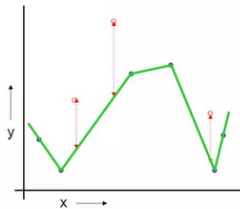
(Linear regression example)

Mean Squared Error = 2.4



(Quadratic regression example)

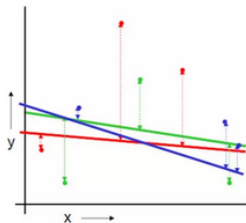
Mean Squared Error = 0.9



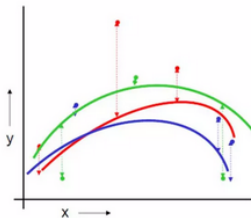
(Join the dots example)

Mean Squared Error = 2.2

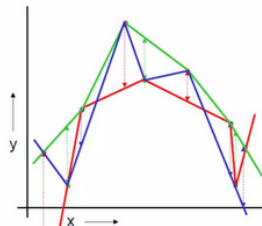
K-Fold Cross-Validation



Linear Regression
 $MSE_{3FOLD}=2.05$



Quadratic Regression
 $MSE_{3FOLD}=1.11$



Joint-the-dots
 $MSE_{3FOLD}=2.93$